

AI PROMPTS & REASONING DOCUMENT

Synapse AI Solutions — Internship Assignment
Multi-Agent Business Research Assistant using LangGraph

Candidate Name	Burhan Aslam
Contact	03212786156
LinkedIn	linkedin.com/in/burhan-aslam/
Role Applied	AI Agent Engineer Intern
Company	Synapse AI Solutions — www.synapseai.us
Assignment	Full Stack Multi-Agent Research Assistant
Primary AI Tool Used	Claude (Anthropic) — claude.ai
Secondary Reference	OpenAI Documentation, LangGraph Documentation
Document Purpose	Transparency log of all AI assistance used during development

1. OVERVIEW & HONEST DECLARATION

This document provides a complete and honest account of how I used AI tools — specifically **Claude by Anthropic** — during the development of the Multi-Agent Business Research Assistant. The assignment explicitly permitted the use of AI tools, stating: *"knowing how to leverage AI effectively is a skill in itself."*

I used Claude as a **learning and development assistant** — to understand frameworks I was learning for the first time (LangGraph, FastAPI), to get code structures I could build upon, and to debug errors as they occurred. Every piece of code was reviewed, understood, tested, and often modified by me before being included in the final project.

What I did independently:

- Designed the overall 4-agent architecture and how agents connect
- Decided routing logic (confidence ≥ 6 threshold, 3-attempt limit)
- Debugged all encoding errors, VPN issues, and library deprecations
- Tested each agent individually before integrating
- Designed the frontend UI layout and user experience flow
- Made all architectural decisions about Full Stack vs Streamlit approach
- Recorded the demo video and prepared all submission materials

2. COMPLETE AI PROMPT LOG

Below is a detailed log of every significant interaction I had with Claude during this project, organized by development phase.

PHASE 1 — Project Understanding & Planning

Prompt P-01	Project Understanding
What I Asked Claude:	
I have received an internship assignment. Please explain what I need to build, what LangGraph is, what multi-agent systems are, and break down the requirements.	
Why I Used AI Here:	
I had never worked with LangGraph or multi-agent systems before. I needed to understand the full scope of the project before writing any code.	
What I Did Myself:	
After Claude explained the concept, I drew my own architecture diagram, understood how agents communicate via shared state, and made the initial project plan.	

Prompt P-02	Architecture Design
What I Asked Claude:	
Explain the complete flow of the 4 agents — Clarity, Research, Validator, Synthesis — how they route to each other and what conditions trigger the next step.	
Why I Used AI Here:	
The routing logic (confidence thresholds, validation loops, max attempt limits) is the core intelligence of the system. I needed to understand every detail.	
What I Did Myself:	
I decided the confidence threshold of 6/10, the 3-attempt maximum for the validation loop, and how the Human-in-the-Loop interrupt fits in. Then I started coding the routing logic.	

Prompt P-03	API Keys Setup
What I Asked Claude:	
How do I get OpenAI and Tavily API keys? What are they used for? How do I store them securely in a Python project?	
Why I Used AI Here:	
First time setting up API keys for a production project. Needed to understand security best practices (.env files, dotenv library).	
What I Did Myself:	
Independently signed up for OpenAI (\$5 credits) and Tavily (free tier). Created the .env file manually with my actual keys.	

PHASE 2 — Backend Development (Agents & Graph)

Prompt P-04	state.py — Shared State Schema
What I Asked Claude:	
Explain what TypedDict is in Python, what Annotated means in LangGraph context, and what add_messages does. Then help me write the Reso	
Why I Used AI Here:	
TypedDict and Annotated are advanced Python concepts I had not used before. The state schema is the foundation everything else builds on —	
What I Did Myself:	
Decided which fields each agent needs, chose appropriate types (Optional[str], Optional[float], int), and understood why messages uses Annotat	

Prompt P-05	tools/tavily_search.py — Web Search Tool
What I Asked Claude:	
Explain what the @tool decorator does in LangChain/LangGraph. Help me write a search tool using Tavily that my Research Agent can call auto	
Why I Used AI Here:	
The @tool decorator and function calling concept were new to me. I needed to understand how an AI agent 'decides' to use a tool.	
What I Did Myself:	
Debugged 3 separate issues myself: (1) UTF-16 encoding error from PowerShell file creation, (2) deprecated TavilySearchResults class — upgr	

Prompt P-06	agents/clarity_agent.py — Agent 1
What I Asked Claude:	
Help me write the Clarity Agent that uses GPT-4 to check if a user's query mentions a specific company name. It should output clarity_status as	
Why I Used AI Here:	
First time using ChatPromptTemplate and StrOutputParser with LangChain chains. Needed the structural pattern to follow.	
What I Did Myself:	
Wrote the system prompt myself defining what 'clear' vs 'ambiguous' means. Chose temperature=0.0 for deterministic decisions. Wrote the respo	

Prompt P-07	agents/research_agent.py — Agent 2
What I Asked Claude:	
Explain bind_tools() in LangGraph. Help me write a Research Agent that automatically decides when to call the Tavily search tool and then eval	
Why I Used AI Here:	
Tool calling (bind_tools) is the most advanced concept in this project. Understanding how GPT-4 autonomously decides to search required caref	
What I Did Myself:	
Designed the two-step process: first call with tools attached (GPT-4 searches), second call for analysis (GPT-4 summarizes). Wrote all system p	

Prompt P-08	agents/validator_agent.py — Agent 3
What I Asked Claude:	
Help me write a Validator Agent that checks research quality with a 3-attempt maximum limit to prevent infinite loops.	
Why I Used AI Here:	
The loop prevention logic (research_attempts counter) needed careful implementation to avoid the workflow getting stuck forever.	

What I Did Myself:
Independently designed the 3 check layers: (1) max attempts force-proceed, (2) empty data check, (3) GPT-4 quality evaluation. Decided 3 as th

Prompt P-09	agents/synthesis_agent.py — Agent 4
What I Asked Claude:	
Help me write the Synthesis Agent that reads conversation history and generates structured emoji-formatted research reports. It must handle fol	
Why I Used AI Here:	
Needed to understand how to pass conversation history to GPT-4 and how HumanMessage/AIMessage objects work in LangChain.	
What I Did Myself:	
Designed the report format (emoji sections) myself. Decided to use last 6 messages for context window management. Chose temperature=0.3 fo	

Prompt P-10	graph.py — Master Workflow
What I Asked Claude:	
Explain StateGraph, add_node, add_edge, add_conditional_edges, MemorySaver, and interrupt() in LangGraph. Help me connect all 4 agents in	
Why I Used AI Here:	
Graph construction with conditional routing and Human-in-the-Loop interrupt is the most complex part of LangGraph. These concepts required d	
What I Did Myself:	
Wrote all 3 routing functions myself (route_after_clarity, route_after_research, route_after_validation). Designed the interrupt_node logic. Chose	

PHASE 3 — Full Stack Development (API + Frontend)

Prompt P-11	main.py — FastAPI Backend
What I Asked Claude:	
Explain what FastAPI is and how it works. Help me build a FastAPI backend with endpoints: POST /chat to start a workflow, POST /chat/clarify to	
Why I Used AI Here:	
First time using FastAPI. Needed to understand REST API concepts, how CORS works, how to run the LangGraph graph inside an API endpoint	
What I Did Myself:	
Decided the API endpoint structure myself. Understood how CORSMiddleware enables frontend-backend communication. Wrote the interrupt de	

Prompt P-12	ui/index.html — Chat Frontend
What I Asked Claude:	
Help me build a professional, modern chat interface in a single HTML file with HTML, CSS, and JavaScript. It should look like a real AI chat app	
Why I Used AI Here:	
Needed guidance on the fetch() API for calling FastAPI endpoints, async/await in JavaScript, and how to build a responsive chat UI with modern	
What I Did Myself:	
Designed the color scheme, layout, and UX flow independently. Wrote the interrupt handling logic — showing clarification questions differently fr	

Prompt P-13	Debugging — CORS Error
What I Asked Claude:	
My frontend is getting a CORS error when calling FastAPI. Explain what CORS is and how to fix it.	
Why I Used AI Here:	
CORS (Cross-Origin Resource Sharing) is a browser security concept I encountered for the first time when connecting frontend to backend.	
What I Did Myself:	
Understood why CORS exists (browser security). Applied CORSMiddleware with correct origins. Verified the fix by testing in browser.	

Prompt P-14	Debugging — VPN & OpenAI Connection
What I Asked Claude:	
My code is getting httpx.ConnectError: getaddrinfo failed when trying to call OpenAI. How do I fix this in Pakistan?	
Why I Used AI Here:	
OpenAI API is not directly accessible from Pakistan. This was a real-world deployment challenge I had to solve.	
What I Did Myself:	
Independently researched and installed Windscribe VPN. Connected to US Central (Atlanta) server. Verified connection before every coding ses	

Prompt P-15	Testing Strategy
What I Asked Claude:	
What 3 test scenarios should I demonstrate in my Loom video to best show all the required features of the assignment?	
Why I Used AI Here:	
Wanted to ensure my video demo covers all assignment requirements — multi-agent flow, Human-in-the-Loop, and multi-turn conversation.	

What I Did Myself:

Chose the specific test queries for each scenario. Planned the video script and flow independently.

3. QUICK REFERENCE SUMMARY

ID	File / Area	AI Used For	I Did Myself
P-01	Project Planning	Explaining LangGraph & multi-agent concepts	Architecture design, agent responsibilities
P-02	System Design	Breaking down 4-agent flow and routing	Confidence thresholds, attempt limits, routing decisions
P-03	API Setup	Guide to getting API keys securely	Signed up, loaded \$5, created .env file
P-04	state.py	TypedDict, Annotated, add_messages concepts	Chose all field names, types, and schema structure
P-05	tavily_search.py	@tool decorator and Tavily integration	Debugged 3 errors, switched to direct API call
P-06	clarity_agent.py	ChatPromptTemplate + chain structure	System prompt wording, temperature choice, parser
P-07	research_agent.py	bind_tools() and tool calling concept	Two-step search+analyze flow, all system prompts
P-08	validator_agent.py	Loop prevention structure	3-layer check design, max attempt decision
P-09	synthesis_agent.py	Conversation history with HumanMessage/AIMessage	3-agent format, 6-message window, temperature
P-10	graph.py	StateGraph, edges, interrupt() concepts	All 3 routing functions, interrupt node logic
P-11	main.py (FastAPI)	FastAPI structure, CORS, endpoint design	API structure decisions, interrupt detection
P-12	index.html (Frontend)	fetch() API, async JS, chat UI structure	Color scheme, UX flow, interrupt UI design
P-13	Debugging CORS	Explaining CORS concept and fix	Applied fix, verified in browser
P-14	Debugging VPN	Identifying OpenAI Pakistan block	Installed Windscribe, configured connection
P-15	Demo Planning	Suggesting test scenarios for video	Chose queries, planned and recorded video

4. WHAT I LEARNED & MY PERSONAL CONTRIBUTIONS

Using AI as a learning tool rather than a shortcut allowed me to genuinely understand the concepts behind everything I built. Here is a summary of the key learnings from this project:

- **LangGraph Multi-Agent Architecture:** I now understand how StateGraph works, how nodes and conditional edges create intelligent routing, and how the shared state enables agent communication. I can design multi-agent systems from scratch.
- **Human-in-the-Loop with interrupt():** I understand how LangGraph's interrupt() pauses a workflow mid-execution, how MemorySaver persists state during the pause, and how Command(resume=...) resumes the workflow. This is an advanced production pattern.
- **Tool Calling / Function Calling:** I understand how bind_tools() gives LLMs the ability to autonomously decide when to call external tools like web search. This is the foundation of agentic AI systems.
- **FastAPI & REST APIs:** I built a complete REST API from scratch with multiple endpoints, CORS handling, and proper request/response models. I understand how the frontend and backend communicate over HTTP.
- **Full Stack Development:** I connected a Python AI backend to a vanilla JavaScript frontend, handling async requests, loading states, and dynamic DOM updates. This is a complete production-grade architecture.
- **Real-World Problem Solving:** I independently solved 4 non-trivial issues: UTF-16 encoding bugs, deprecated library migration, Tavily API response format changes, and VPN setup for regional API restrictions.

My Honest Assessment

AI tools made me significantly more productive — especially for understanding new frameworks like LangGraph and FastAPI. However, the **thinking, designing, debugging, and decision-making** was mine throughout. Every error I encountered (and there were many), I diagnosed myself before asking for help. Every architectural decision — the 4-agent structure, the routing thresholds, the Full Stack approach over Streamlit — was my own judgment call.

I believe this is exactly the kind of AI-augmented development that modern engineering requires — not replacing thinking, but accelerating learning and execution.

Submitted by: Burhan Aslam | 03212786156 | [linkedin.com/in/burhan-aslam/](https://www.linkedin.com/in/burhan-aslam/)

Synapse AI Solutions — AI Agent Engineer Internship Assignment